

```

import PySide6.QtWidgets
from PySide6.QtWidgets import QStyledItemDelegate
from PySide6.QtGui import QColor, QFont, Qt, QCursor, QPalette
from PySide6.QtCore import QEvent

class IndexLinkDelegate(QStyledItemDelegate):
    """Styles IDn reference columns as underlined web hyperlinks with a hover hand cursor."""

    def paint(self, painter, option, index):
        # Column 0 is reserved for structural heading text; style only IDn columns
        if index.column() == 0:
            super().paint(painter, option, index)
            return

        # Create a copy of the layout style configuration to modify safely
        custom_option = PySide6.QtWidgets.QStyleOptionViewItem(option)
        self.initStyleOption(custom_option, index)

        # Apply a web hyperlink blue color map palette
        custom_option.palette.setColor(QPalette.ColorRole.Text, QColor(0, 102, 204))
        custom_option.palette.setColor(QPalette.ColorRole.HighlightedText, QColor(255, 255, 255))

        # Inject an underline property directly onto the text font metrics configuration
        font = custom_option.font
        font.setUnderline(True)
        custom_option.font = font

        # Hand rendering operations back over to the core Qt styling engine
        super().paint(painter, custom_option, index)

    def editorEvent(self, event, model, option, index):
        """Monitors mouse positions via style options to toggle the pointing hand cursor on hover."""
        # Only adjust cursor indicators over horizontal reference link fields (Column 1+)
        if index.column() > 0:
            # CRITICAL REPAIR: Retrieve target viewport context out of the style option parameter
            view = option.widget

            if view:
                if event.type() == QEvent.Type.MouseMove:
                    # Check if the text coordinate model index under the cursor is valid data
                    if index.isValid():
                        view.setCursor(QCursor(Qt.CursorShape.PointingHandCursor))
                    else:
                        view.setCursor()

                elif event.type() == QEvent.Type.Leave:
                    # Reset immediately if the mouse sweeps outside the view bounds
                    view.setCursor()

        return super().editorEvent(event, model, option, index)

```